AI Usage

We were encouraged to use AI tools (ChatGPT, Claude, Gemini, Cursor, Copilot) for this assignment. Below are the meaningful AI-assisted decisions, what was accepted or rejected, and why.

---

1. Replacing the `uuid` Library with `crypto.randomUUID()`

AI suggestion (Claude): Node.js v22 has `crypto.randomUUID()` built-in, which is simpler than adding the `uuid` npm package, especially since uuid v14 is ESM-only and breaks CommonJS Jest.

Accepted: Yes. Replaced `require('uuid')` with `require('crypto').randomUUID()` across all 5 domain model files. This eliminated an external dependency and removed the Jest ESM-in-CommonJS issue that was causing all tests to fail.

Why: The uuid library introduced ESM/CJS compatibility issues with Jest. Using the built-in `crypto.randomUUID()` is zero-dependency, natively CommonJS compatible, and semantically equivalent.

---

2. Feedback Scoring Algorithm Design (Rule-Based Provider)

AI suggestion (Claude): To make deterministic feedback objective and reproducible, base scores on concrete heuristics: count class declarations for modularity, check for keyword mentions of SOLID principles/patterns, and evaluate explanation quality for trade-off analysis.

Accepted (with modifications): Adopted the heuristic approach but rejected overly strict thresholds. Initially, the AI suggested hard thresholds like "3+ classes = score 5", which would penalize good designs with fewer classes. Modified to use a sliding scale: minimum 2 classes with 5+ methods for a top modularity score, with a floor tied to class count.

Why: Pure keyword matching is imperfect but serves as a deterministic baseline. The LLM-based provider adds nuance on top.

---

3. Feedback Provider Chain of Responsibility Pattern

AI suggestion (Claude): Use a Chain of Responsibility pattern for feedback providers, where the rule-based provider runs first (fast, deterministic), and the LLM provider acts as a fallback. If the LLM API fails or is unavailable, the system still returns deterministic feedback.

Accepted: Yes, implemented `FeedbackProviderChain`. However, modified to run both providers and merge results, rather than short-circuiting on the first provider. This gives richer feedback by combining deterministic structural checks with LLM qualitative insights.

Why: Combining both sources of feedback provides more value than a simple fallback chain. The merge approach also naturally handles the question of "which parts should be deterministic vs LLM."

---

4. Domain-Driven Folder Structure (Clean Architecture)

AI suggestion (Claude): Organize the codebase into domain/application/infrastructure/presentation layers, following Clean Architecture and Domain-Driven Design principles. Domain entities should be independent of infrastructure concerns.

Accepted: Yes, but adapted for a prototype. The initial suggestion included separate interface files for repositories, which I accepted since it demonstrates the Dependency Inversion Principle clearly — the application layer depends on domain interfaces, not concrete infrastructure implementations.

Not accepted: The AI also suggested a more complex event-sourcing pattern for attempt history. Rejected this as over-engineering for a 2-day assignment focused on LLD practice, not distributed systems.

Why: Clean architecture makes the LLD principles (the assignment's focus) visible in the code structure itself, which is exactly what evaluators should look for.

---

5. LLM Feedback Simulation for Prototype

AI suggestion (Claude): Since we may not have API access during evaluation, implement a simulation mode in `LLMBasedFeedbackProvider` that generates plausible LLM-style responses without making actual API calls.

Accepted: Yes. The `LLMBasedFeedbackProvider` uses `apiKey: null` by default, which triggers `_simulateLLM()` — a delayed response that produces realistic structured feedback. When an `OPENAI_API_KEY` is provided, it calls the real OpenAI API.

Why: Makes the prototype self-contained and demonstrable without requiring API credentials, while keeping the pathway to real LLM evaluation.

Summary

#	Decision	AI Tool	Accepted?
1	Replace uuid with crypto.randomUUID()	Claude	Yes
2	Rule-based scoring heuristics	Claude	Partially (modified thresholds)
3	Feedback provider chain/merge	Claude	Yes (modified to merge both)
4	Clean architecture layering	Claude	Yes (rejected event sourcing)
5	LLM feedback simulation	Claude	Yes